

Activity 13 - Document an API with rustdoc and doctests

AI Vibe Coding with Rust / TGS-2023039924 / version 3.0 / K5/A5

Specification

`reorder_point(daily_usage, lead_days, safety_stock) -> Option<u32> = daily_usage x lead_days + safety_stock` (R1), None on overflow (R2). `needs_reorder(on_hand, point)` is true at or below the point (R3). Every public item is documented; cargo clippy with `-D warnings` enforces `missing_docs`.

Plan before code

Act as a senior Rust reviewer. Specification: `reorder_point(daily_usage, lead_days, safety_stock) -> Option<u32> = daily_usage x lead_days + safety_stock` (R1), None on overflow (R2). `needs_reorder(on_hand, point)` is true at or below the point (R3). Every public item is documented; cargo clippy with `-D warnings` enforces `missing_docs`. Before writing any code, list the public signatures, the edge cases and at least four boundary tests you would expect. Ask me about anything ambiguous.

Generate with AI

Implement "Document an API with rustdoc and doctests" in `starter/src` so that the supplied tests pass. Ask the AI to draft docs from the code and requirements only; verify every example by running cargo test. Keep public signatures unchanged, use no unsafe code, and explain each design decision in one sentence.

Repair from evidence

Here is my exact cargo output: `<paste the first error or failing test>`. Explain the cause in plain English, propose the smallest fix that keeps the specification, and add one regression test. Do not change expected values in existing tests.

Review checklist

Signatures unchanged; no unsafe; no `unwrap()` on input; no new crates; every boundary tested; cargo test, clippy `-D warnings` and `fmt --check` pass; you can explain every line.